—-------ÖZET—--------

AI-Assisted Root Cause Analysis for DevOps & SRE Runbook Automation

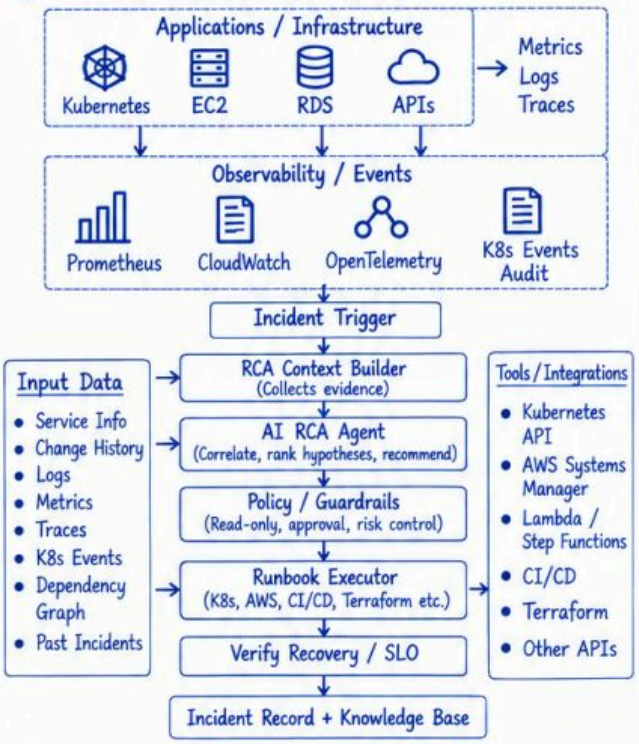
JyothiMulkuntla ♡

Goal: Faster incident resolution with AI-assisted diagnosis, controlled automation and safe, reliable operations.

① End-to-End Flow

Alert → Collect Evidence → AI RCA (Correlate & Analyze) → Runbook (Recommend) → Human Approval → Execute → Validate Recovery → Learn & Improve

② Reference Architecture



③ Example: Kubernetes RCA

Alert: KubePodCrashLooping
namespace = payments
pod = payment-api-7b94...

Collected Evidence:
- kubectl describe pod
- kubectl logs --previous
- kubectl get events
- Deployment & rollout history
- Resource usage

RCA Result:
- **Cause:** Container memory limit exceeded
- **Confidence:** High
- **Evidence:** OOMKilled, peak memory > 512Mi
- **Recommendation:** Rollback to v4.7.0
- **Risk:** Medium
- **Approval:** Required

④ Runbook Example (Database)

PostgreSQL Connection Exhaustion

- ☐ **Conditions:**
  - connection_utilization > 90%
  - application_connection_timeout = true
- ☐ **Automatic Actions:**
  - collect_pg_stat_activity
  - identify_connection_source
- ☐ **Approval Required:**
  - terminate_sessions
  - restart_database
  - resize_database
- ☐ **Validation:**
  - connection_utilization < 70%
  - application_error_rate < 1%

Turn documentation into executable operational knowledge!

⑤ Action Risk Levels

Action	Automation
Query pods / logs / metrics	Automatic
Query DB statistics	Automatic
Restart stateless pod	Policy-dependent
Scale deployment	Approval
Roll back application	Approval recommended
Kill DB connections	Human approval
Modify security group/IAM	Human approval
Delete resources	Never unrestricted

⑥ AWS Implementation (2026)

Use Amazon Bedrock AgentCore (AgentCore Runtime + Gateway)

- EventBridge / Alertmanager
- CloudWatch / OpenSearch
- K8s API / CloudTrail
- GitHub / Deployment data
- AgentCore Gateway (controlled tools)
- Policy engine + Human approval

⑦ Guardrails & Best Practices

- ☑ Least-privilege access
- ☑ Read-only default
- ☑ Allowlisted operations
- ☑ Human approval for high impact
- ☑ Immutable audit trail
- ☑ Runbook versioning (Git)
- ☑ Evidence citations
- ☑ Confidence thresholds
- ☑ Blast-radius limits
- ☑ Post-action validation
- ☑ Auto abort/rollback on failures

⑧ Recommended Phases (MVP → Production)

1. Incident summary (collect telemetry)
2. Evidence correlation & RCA hypotheses
3. Retrieve runbook & recommend commands
4. Execute read-only diagnostics
5. Human-approved remediation
6. Policy-based auto-remediation (low-risk)

⑨ Target Outcome

- ☑ Reduced MTTR
- ☑ Consistent runbook execution
- ☑ Better signal correlation
- ☑ Safe & controlled automation
- ☑ More reliable systems

"Automate with Intelligence, Operate with Confidence."

JyothiMulkuntla ♡

"Reliable Systems Brighter Tomorrows."

The Most Underrated DevOps Skill

It isn't Kubernetes. It isn't Terraform. It isn't Docker.
It's Troubleshooting.

A DevOps engineer can know many tools, but when production breaks, the important question is:

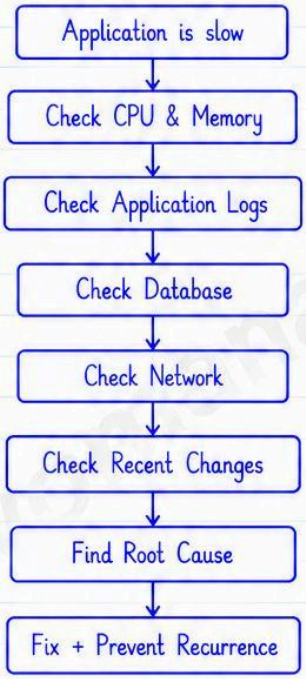
"Can you find out why it broke?"

1. What Good Troubleshooting Looks Like

Imagine your application suddenly becomes slow.
A beginner might say:

"Restart the Pod."

An experienced engineer starts investigating:



2. The Skill Behind the Tools

Troubleshooting means knowing how to:

- Read logs
- Understand errors
- Check metrics
- Trace requests
- Test network connectivity
- Compare before/after changes
- Find the root cause
- Verify the fix

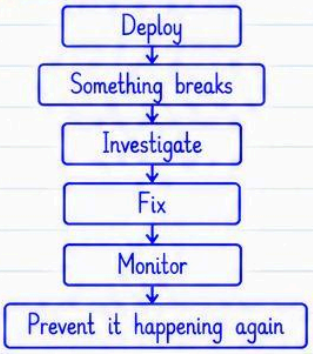
You don't need to immediately know the answer.
You need to know how to find the answer.

3. Why It Matters

DevOps is not just about:

Build → Deploy → Done

Real environments look more like:



4. Easy Memory Trick

Don't memorize every solution. Learn how to investigate the problem.

Strong troubleshooting skills can make you effective even when you're working with a tool you've never used before.

www.vyomanant.com | academy@vyomanant.com | +91 9704286195

Shift-Left Security

Vyomanant academy

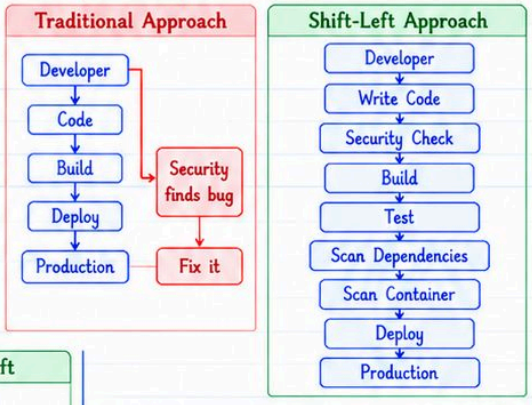
Useful Practice or Just Another Buzzword?

1. Short Answer

It is useful — but only when security is actually integrated into the development process.

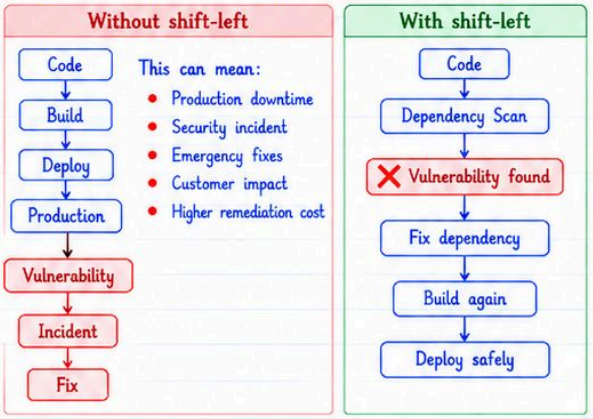
"Shift left" means finding and fixing security problems earlier, instead of waiting until the application is already in production.

2. Simple Example



3. Why does "Shift Left" matter?

Imagine a developer accidentally adds a vulnerable library.



4. What security checks can move left?

Security doesn't mean only one scanner.
You can introduce checks at different stages:

Stage	Example Security Check
Code	SAST / code scanning
Dependencies	Vulnerable package detection
Secrets	API key/password detection
Docker Build	Container image scanning
IaC	Terraform/Kubernetes config scanning
CI/CD	Pipeline security controls
Deployment	Policy checks
Production	Runtime monitoring

5. Example with Docker



Suppose the image contains an outdated vulnerable OpenSSL package. A container scanner can detect it before the image reaches production. That is shift-left security.

6. Important: What shift-left is NOT

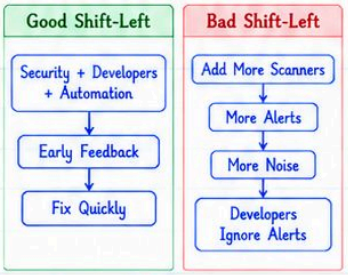
Shift-left does NOT mean:

"Add 10 security scanners to the CI/CD pipeline."

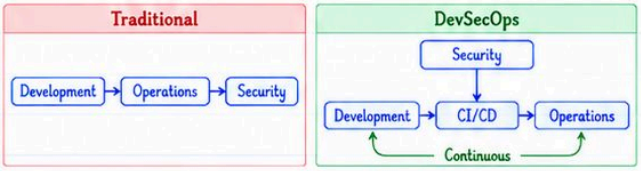
This can become a buzzword if every small commit gets blocked by:

- Too many false positives
- Slow scans
- Unclear security policies
- Hundreds of irrelevant findings
- Developers may simply start ignoring security results.

7. Good vs Bad Shift-Left



8. DevSecOps Connection



9. Easy Memory Trick

Shift Left = Security Earlier

Instead of:

Build → Deploy → Find Security Problem

Do:

Code → Check Security → Fix → Deploy

10. One-Sentence Summary

Shift-left security is not just a buzzword when it gives developers fast, automated security feedback early enough to fix problems before production.

www.vyomanant.com | academy@vyomanant.com | +91 9704286195

DevSecOps : Integrating SonarQube, Trivy & Checkov

Shift Security Left in CI/CD Pipeline

"Secure Code → Secure Build → Secure Infrastructure"

1) What is DevSecOps ?

- DevSecOps means integrating Security into DevOps.
- Find and fix security issues early in the development lifecycle.
- Automate security checks in CI/CD pipeline.
- **Goal :** Faster, safer and more reliable deployments.

2) Tools Overview

SonarQube

- Static Code Analysis (SAST)
- Finds code quality issues & security vulnerabilities
- Supports multiple languages (Java, Python, etc.)
- Provides quality gate

Trivy

- Container Image Scanning
- Finds OS packages and application vulnerabilities
- Integrates with CI/CD
- Lightweight & easy to use

Checkov

- Infrastructure as Code (IaC) Scanning
- Scans Terraform, CloudFormation, Kubernetes, etc.
- Finds misconfigurations
- Helps to follow security best practices

Key Benefits

- ✓ Early vulnerability detection
- ✓ Improved code quality
- ✓ Secure container images
- ✓ Secure infrastructure configurations
- ✓ Automated security in CI/CD
- ✓ Compliance with best practices
- ✓ Reduced risk in production

3) CI/CD Pipeline with Security Integration



Result

- ✓ Vulnerabilities caught early
- ✓ Secure builds and images
- ✓ Policy compliant infrastructure
- ✓ Smooth and safer deployments

4) Example : Jenkins Pipeline (Simplified)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git 'https://github.com/example/app.git' }
    }
    stage('SonarQube Scan') {
      steps { sh 'sonar-scanner' }
    }
    stage('Build Docker Image') {
      steps { sh 'docker build -t myapp:${BUILD_NUMBER} .' }
    }
    stage('Trivy Scan') {
      steps { sh 'trivy image --exit-code 1 myapp:${BUILD_NUMBER}' }
    }
    stage('Checkov Scan') {
      steps { sh 'checkov -d terraform/ --framework terraform' }
    }
    stage('Deploy') {
      steps { sh 'kubectl apply -f k8s/' }
    }
  }
}
```

Pipeline fails if any high/ critical issue is found.

5) Tool Commands

SonarQube

```
# Install & Run Scanner
sonar-scanner \
  -Dsonar.projectKey=myapp \
  -Dsonar.sources=. \
  -Dsonar.host.url=http://sonarqube:9000 \
  -Dsonar.login=TOKEN
```

Trivy

```
# Scan Docker Image
trivy image myapp:latest

# Fail on high/critical
trivy image --exit-code 1 \
  --severity HIGH,CRITICAL \
  myapp:latest
```

Checkov

```
# Scan Terraform code
checkov -d terraform/

# Output as SARIF (optional)
checkov -d terraform/ \
  --output sarif \
  --output-file-path \
    checkov-results.sarif
```

6) Real-time Use Case (from my project)

- ✓ Used SonarQube to maintain code quality and security for Java and Python microservices.
- ✓ Integrated Trivy to scan Docker images before pushing to ECR.
- ✓ Used Checkov to validate Terraform modules for AWS infrastructure (VPC, EKS, IAM, etc.).
- ✓ Implemented in Jenkins CI/CD pipeline with quality gates.
- ✓ Reduced security issues in production and followed organization security compliance.

7) Key Takeaways

- ✓ Integrate security tools in CI/CD from day one.
- ✓ Use quality gates and fail builds on critical issues.
- ✓ Scan code, containers and infrastructure.
- ✓ Automate and make it part of the pipeline.
- ✓ Security is everyone's responsibility (Dev + Ops + Security).

"Build Secure Deploy Confidently"

DevSecOps = Secure + Reliable + Faster + Better Operations

Code | Scan | Secure | Deploy | Repeat

Keep Learning Keep Building

DOCKER CHEAT SHEET • 2026 •

HOW DOCKERS ACTUALLY WORKS?

BUILD ONCE. RUN ANYWHERE. ISOLATED. PORTABLE. CONSISTENT.

Docker packages your app and all its dependencies into a container.

01 WRITE DOCKERFILE

Dockerfile

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```

What it is:
A blueprint with instructions to build your image.

02 BUILD IMAGE

Result:
Docker builds the image in layers and bundles everything.

03 IMAGE (READ-ONLY)

What it contains:
- App code
- Dependencies
- Libraries
- Runtime
- Configs

04 RUN CONTAINER

What it gets:
- Isolated filesystem
- Its own network
- CPU / Memory
- Process space

05 SHARE (REGISTRY)

What happens:
Push image to a registry (Docker Hub/private) and pull anywhere.

06 RUN ANYWHERE

Benefit:
Same image runs on laptop, server, cloud — consistently.

DOCKER ARCHITECTURE

CLIENT
- Docker CLI
- Docker Desktop

DOCKER DAEMON (DOCKER ENGINE)
- Images
- Containers
- Networks
- Volumes

DOCKER ENGINE
- REST API
- Swarm Manager
- Image Builder
- Container Runtime

HOST OS

INFRASTRUCTURE
Server / VM / Cloud

REGISTRY
- Docker Hub
- Private Registry

CONTAINER LIFECYCLE

Image (Pulled/Local) → Create Container → Start Container → Running (Your App) → Stop Container → Remove Container

CONTAINER VS VIRTUAL MACHINE

CONTAINER		VIRTUAL MACHINE
✓ Lightweight	VS	✗ Heavyweight
✓ Fast start		✗ Slower
✓ High density		✗ Lower density
✓ Shares host OS kernel		✗ Needs full OS

UNION FILE SYSTEM (LAYERED)

Each instruction in Dockerfile creates a layer.
Layers are cached & reused.
Efficient & fast builds.

VOLUMES & DATA

Container Writable Layer
Volume (Persistent)

Keeps data separate from container.
Data survives container restart or removal.

NETWORKING

Each container gets its own network stack.
Containers communicate using names or IPs.

COMMON DOCKER COMMANDS

`docker build -t myapp .`	Build image from Dockerfile
`docker images`	List local images
`docker run -d --name myapp -p 8080:80 myapp`	Run container
`docker ps`	List running containers
`docker logs myapp`	View container logs
`docker stop myapp`	Stop container
`docker rm myapp`	Remove container
`docker rmi myapp`	Remove image
`docker push myapp`	Push image to registry
`docker pull myapp`	Pull image from registry

DOCKER IN ACTION (EXAMPLE FLOW)

Developer → Build Image → Push to Registry → Pull on Server → Run Container

✓ Consistent Environment ✓ Easy Deployment ✓ Scalable ✓ Portable ✓ Reliable

WHY DOCKER?

- ✓ Consistent across environments
- ✓ Isolated & secure applications
- ✓ Quick startup & high performance
- ✓ Easy to ship, share & scale
- ✓ Works everywhere

BEST PRACTICES

- ✓ Use minimal base images
- ✓ Keep images small
- ✓ Don't run as root
- ✓ Scan images for vulnerabilities
- ✓ Use .dockerignore
- ✓ Update base images regularly

REMEMBER

Docker doesn't run your app. It packages your app with everything it needs and runs it in an isolated container. You build it. Docker ships it. **Anywhere, anytime!**

Oolooroo.io

Dockerfile Instructions

- A Dockerfile is a text file containing instructions Docker uses to build a container image.
- Instructions are executed from top to bottom, with each build step creating a cacheable layer.

JyothiMulkuntla

Instruction	Purpose	Example
FROM	Defines the base image	FROM ubuntu:24.04
LABEL	Adds image metadata	LABEL maintainer= "admin@example.com"
RUN	Executes commands during image build	RUN apt-get update && apt-get install -y curl
WORKDIR	Sets the working directory	WORKDIR /app
COPY	Copies files from build context into image	COPY . /app
ADD	Copies files with additional archive/URL-related behavior	ADD app.tar.gz /app/
ENV	Defines environment variables	ENV APP_ENV=production
ARG	Defines build-time variables	ARG VERSION=1.0
EXPOSE	Documents the port used by inte application	EXPOSE 8080
USER	Specifies the user for subsequent commands/runtime	USER appuser
CMD	Defines the default container command/arguments	CMD ["python", "app.py"]
ENTRYPOINT	Defines the primary executable	ENTRYPOINT ["java", "-jar", "app.jar"]
VOLUME	Declares a mount point	VOLUME ["/data"]
HEALTHCHECK	Defines container health checking	HEALTHCHECK CMD curl -f http://localhost:8080/health || exit 1
USELL	Changes the default shell	SHELL ["/bin/bash", "-c"]
STOPSIGNAL	Specifies the signal used to stop the container	STOPSIGNAL SIGTERM
ONBUILD	Adds instructions executed when another image uses this as a base	ONBUILD COPY . /app

Build Flow

1. Base Image (FROM)
2. Working Directory (WORKDIR)
3. Dependencies (RUN)
4. Application Files (COPY/ADD)
5. Port (EXPOSE)
6. Startup Command (CMD / ENTRYPOINT)

Easy Memory Rule

FROM → RUN → WORKDIR → COPY → ENV → EXPOSE → USER → CMD / ENTRYPOINT

Small steps
Big Images
Happy Containers !

RUN vs CMD vs ENTRYPOINT

- **RUN** → runs while the image is being built.
- **CMD** → supplies the default command or arguments when the container starts.
- **ENTRYPOINT** → defines the container's main executable.

RUN (Build time) → CMD (Default) → ENTRYPOINT (Main)

COPY vs ADD

- **COPY** → use for normal file copying.
- **ADD** → use only when you need additional behavior (e.g., automatic extraction of a local tar archive or URL support).

COPY app.py /app/

Common Dockerfile Example

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "app.py"]
```

Production Best Practices

- ✓ Use small, trusted base images
- ✓ Pin important versions
- ✓ Combine related commands to improve caching
- ✓ Use .dockerignore
- ✓ Avoid storing passwords or API keys in the Dockerfile
- ✓ Run applications as a non-root user

Production Example (Non-root user)

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["python", "app.py"]
```

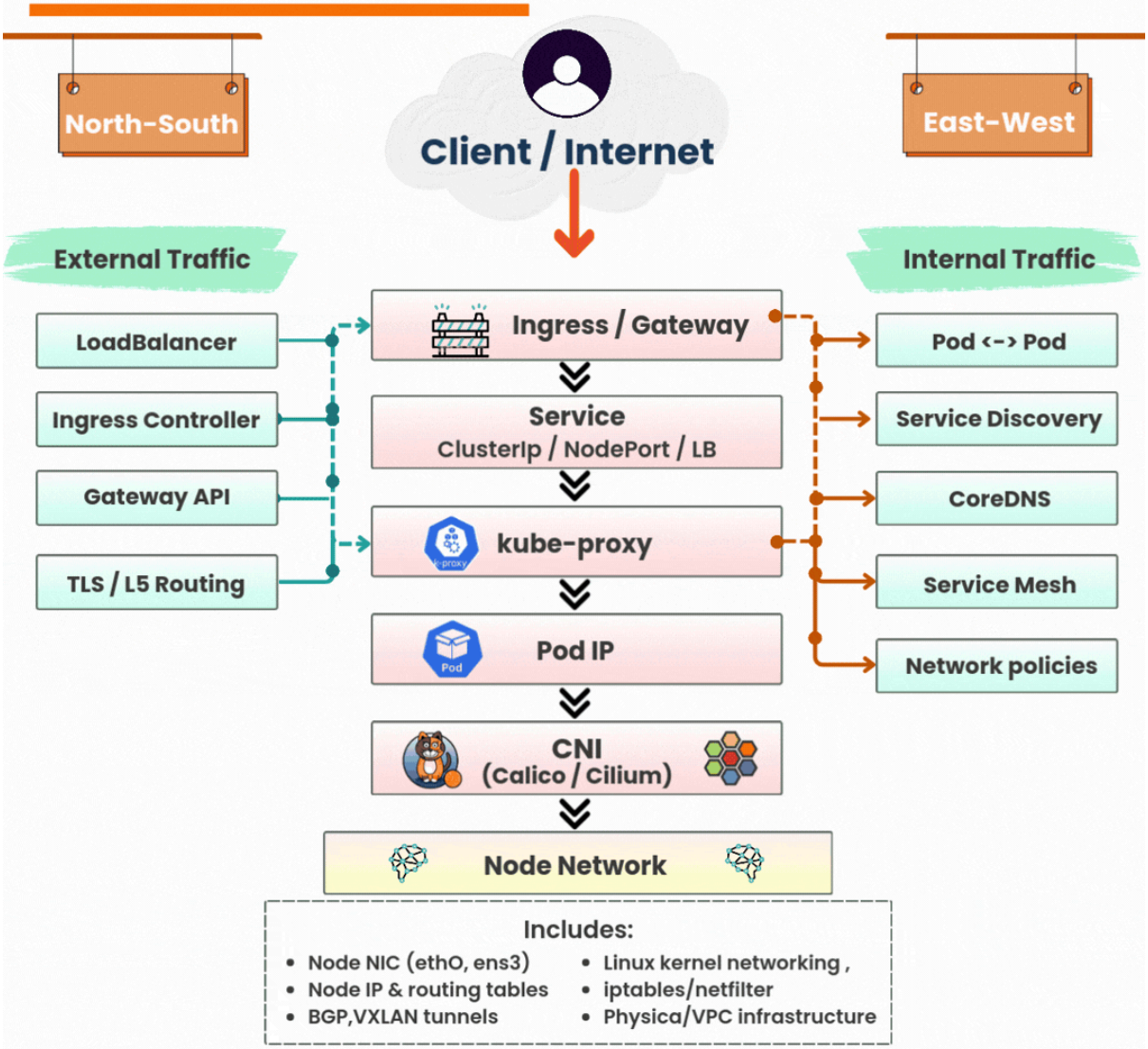
Build
Ship
Run

Better Containers
Brighter Futures !

JyothiMulkuntla

How Traffic Flows In Kubernetes

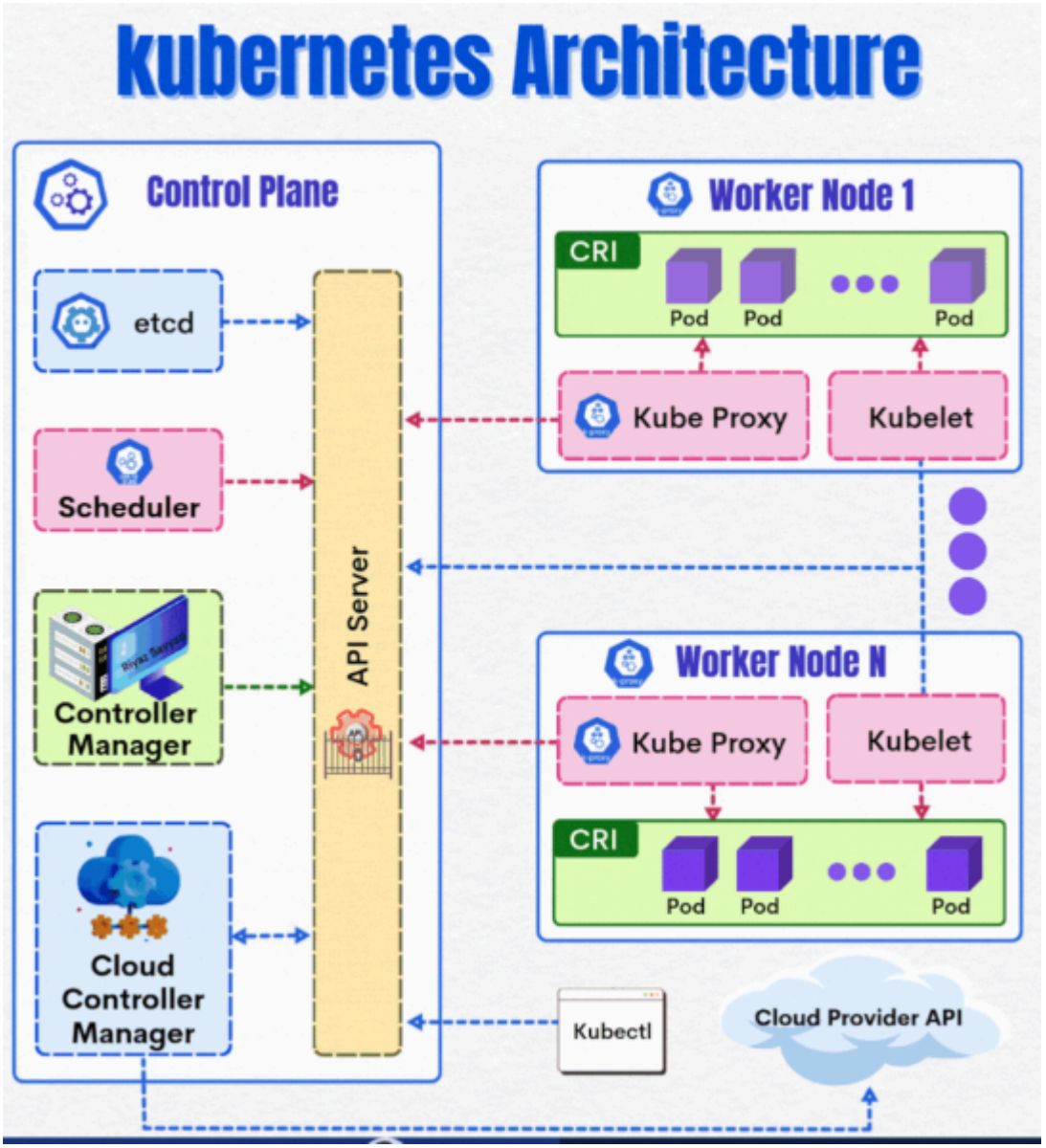
Vishakha Sadhwani
@vsadhwani

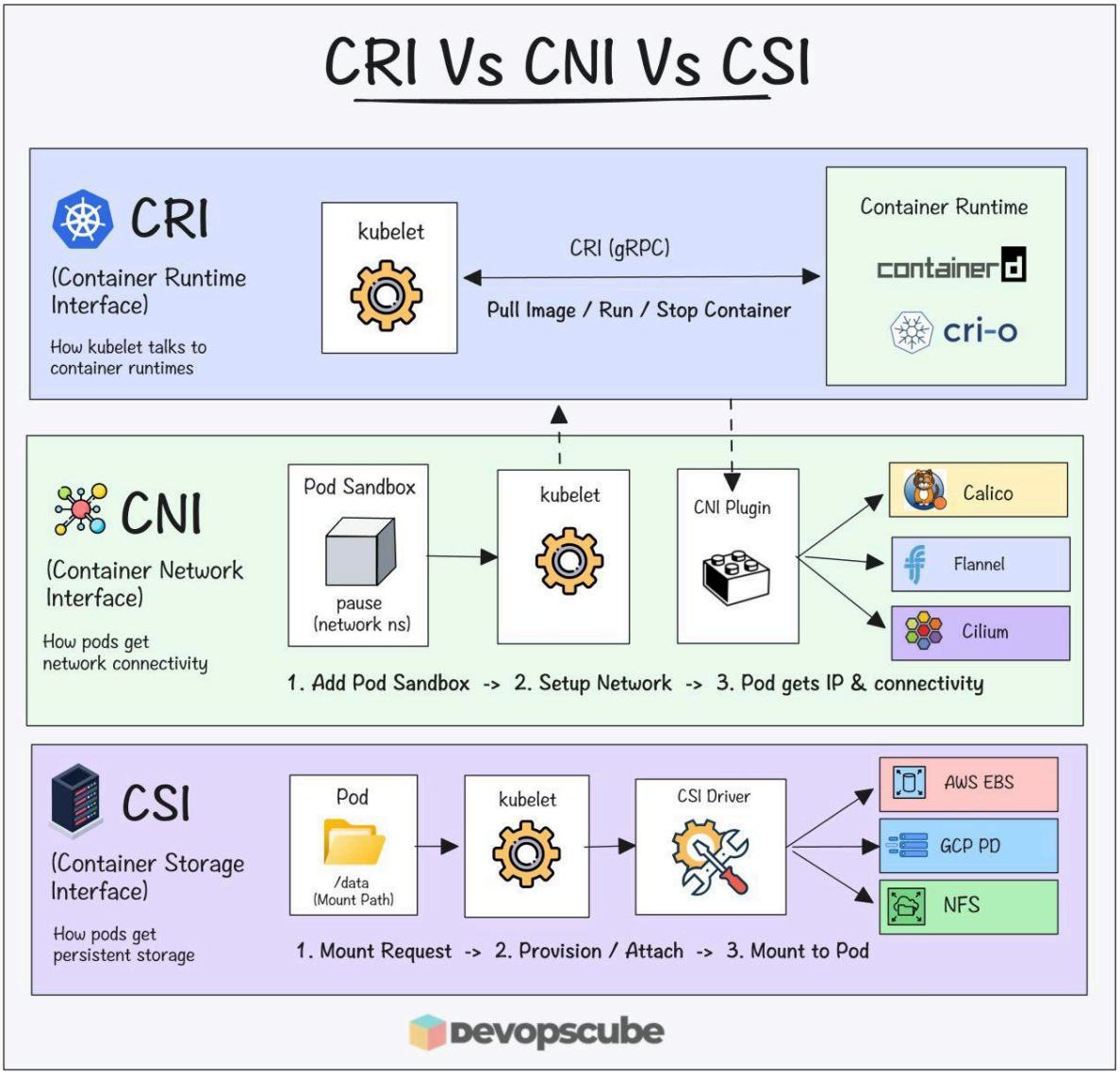


Foundation

- CNI (Calico / Cilium)
- Network Policies
- Flat network model
- IP-per-Pod architecture
- No NAT: Pod IP model
- IPAM by CNI
- Service abstraction layer
- Overlay networking (VXLAWBGP)
- All Pods routable
- East-West: Pod-to-Pod direct
- North-South: via Ingress/Gateway
- DNS-based service discovery

Traffic: North-South (External <-> Cluster) | East-West {Pod <-> Pod within Cluster)


kubernetes Architecture
Control Plane
etcd
Scheduler
Controller Manager
Cloud Controller Manager
API Server
Worker Node 1
CRI
Pod
Pod
Pod
Kube Proxy
Kubelet
Worker Node N
Kube Proxy
Kubelet
CRI
Pod
Pod
Pod
Kubectl
Cloud Provider API


CRI Vs CNI Vs CSI
CRI
(Container Runtime Interface)
How kubelet talks to container runtimes
kubelet
CRI (gRPC)
Pull Image / Run / Stop Container
Container Runtime
containerd
cri-o
CNI
(Container Network Interface)
How pods get network connectivity
Pod Sandbox
pause (network ns)
kubelet
CNI Plugin
Calico
Flannel
Cilium
1. Add Pod Sandbox -> 2. Setup Network -> 3. Pod gets IP & connectivity
CSI
(Container Storage Interface)
How pods get persistent storage
Pod
/data (Mount Path)
kubelet
CSI Driver
AWS EBS
GCP PD
NFS
1. Mount Request -> 2. Provision / Attach -> 3. Mount to Pod
devopscube

StatefulSet vs Deployment

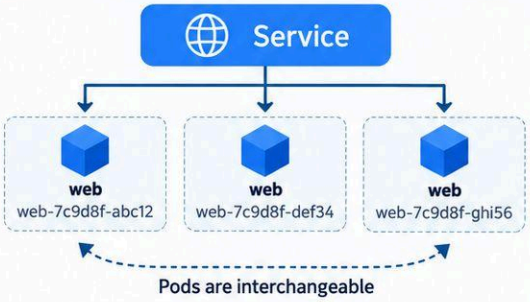
When Should You Use Each?

SAME PLATFORM. DIFFERENT NEEDS. A STRONGER CLOUD-NATIVE TOMORROW.

BUILD
DEPLOY
SCALE

Deployment

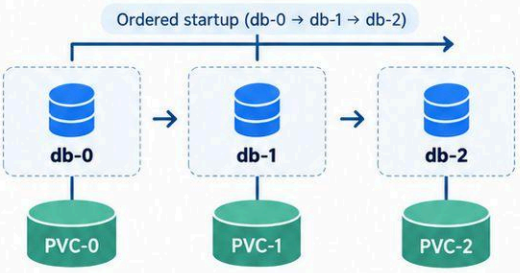
Stateless apps, built for scale



- Best for stateless applications
- Pods are interchangeable
- Random pod identities
- Easy scaling and rollouts
- Typical use cases: web apps, APIs, microservices

StatefulSet

Stateful apps, with stable identity



- Best for stateful applications
- Stable pod identities
- Persistent volumes per pod
- Ordered deployment and termination
- Typical use cases: PostgreSQL, MySQL, MongoDB, Kafka

Key Difference

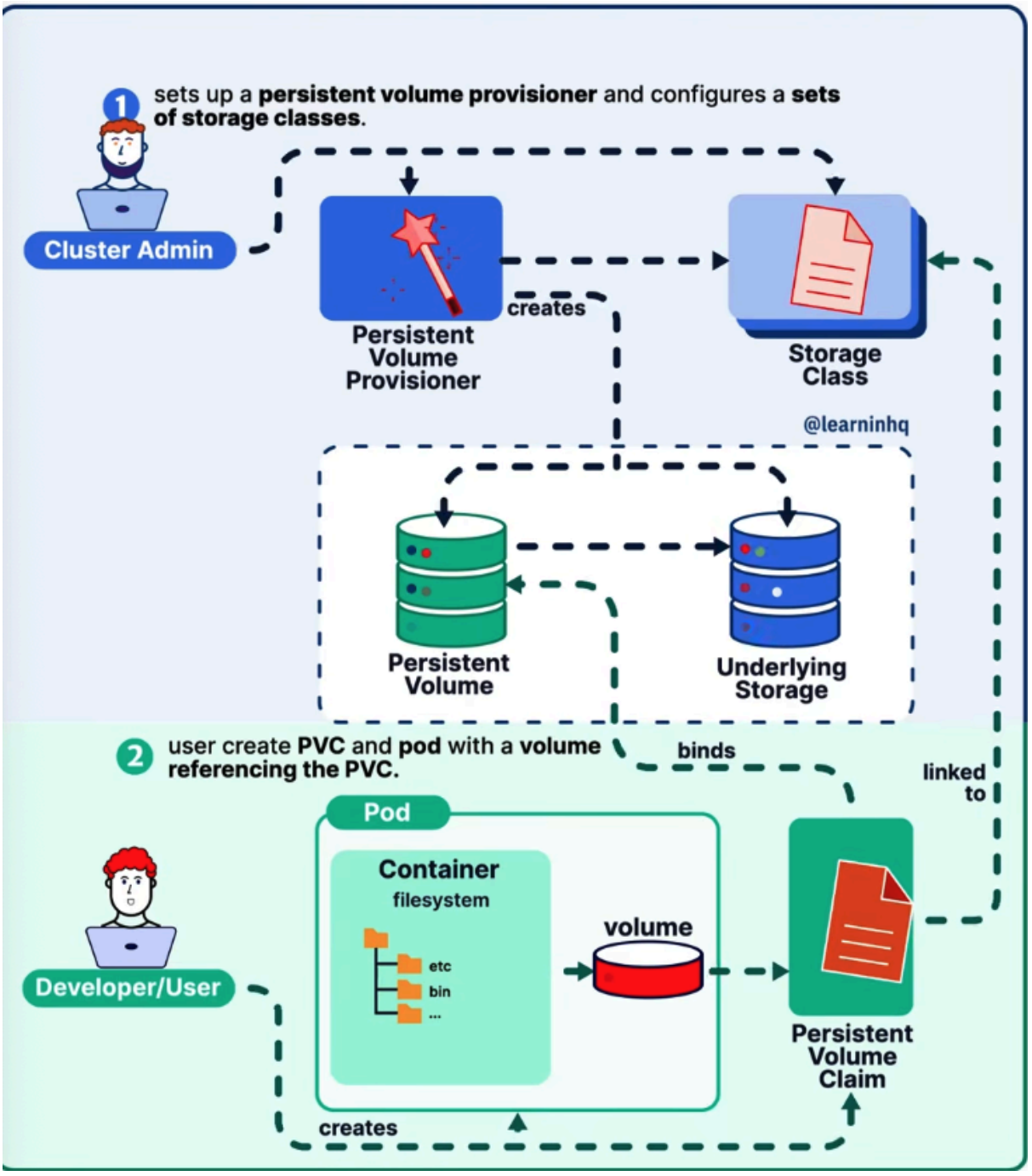
Stateless: app does not care which pod serves the request.

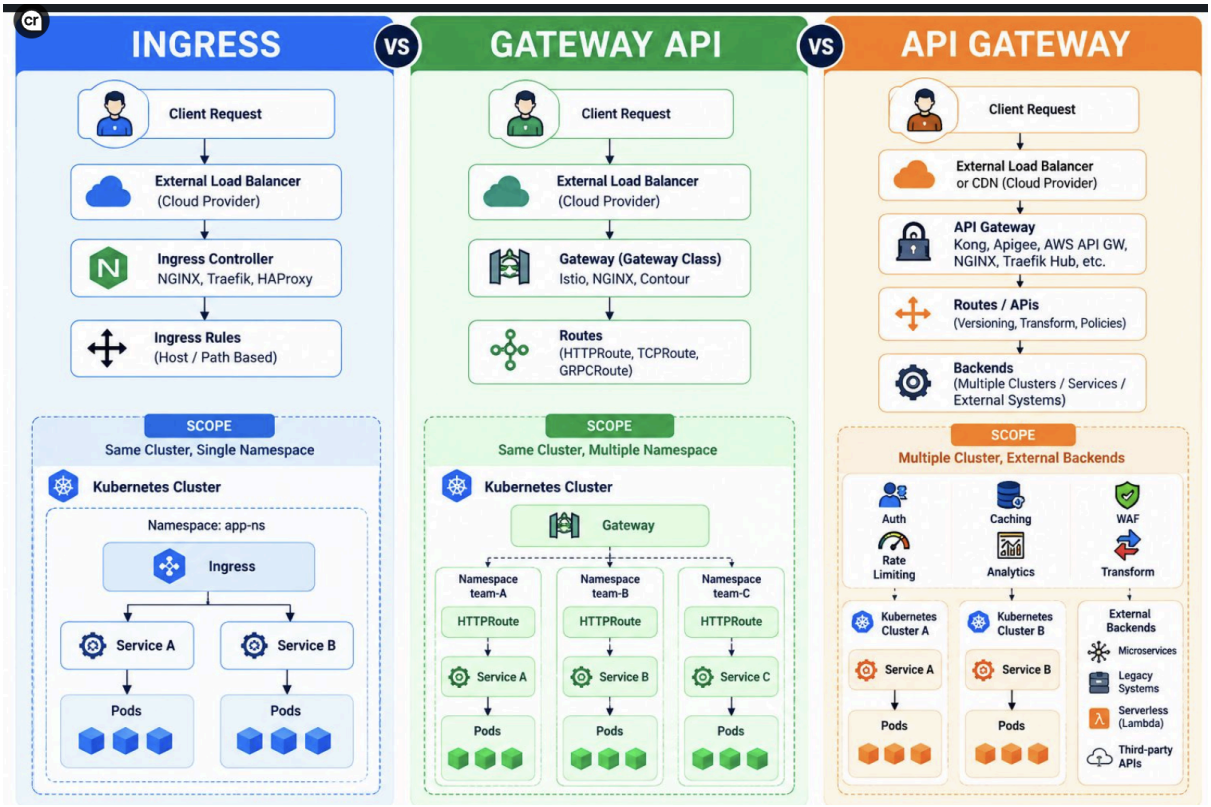
Stateful: app needs stable identity and durable storage.

Databases on Kubernetes

Use StatefulSet when identity, storage, and startup order matter.

RELIABLE
APPLICATIONS
BRIGHTER TOMORROW


1
sets up a **persistent volume provisioner** and configures a **sets of storage classes**.
Cluster Admin
Persistent Volume Provisioner
creates
Storage Class
@learninhq
Persistent Volume
Underlying Storage
2
user create **PVC** and **pod** with a **volume** referencing the PVC.
binds
linked to
Pod
Container
filesystem
etc
bin
...
volume
Developer/User
Persistent Volume Claim
creates


INGRESS
VS
Client Request
External Load Balancer
(Cloud Provider)
Ingress Controller
NGINX, Traefik, HAProxy
Ingress Rules
(Host / Path Based)
SCOPE
Same Cluster, Single Namespace
Kubernetes Cluster
Namespace: app-ns
Ingress
Service A
Service B
Pods
Pods
GATEWAY API
VS
Client Request
External Load Balancer
(Cloud Provider)
Gateway (Gateway Class)
Istio, NGINX, Contour
Routes
(HTTPRoute, TCPRoute,
GRPCRoute)
SCOPE
Same Cluster, Multiple Namespace
Kubernetes Cluster
Gateway
Namespace
team-A
Namespace
team-B
Namespace
team-C
HTTPRoute
HTTPRoute
HTTPRoute
Service A
Service B
Service C
Pods
Pods
Pods
API GATEWAY
Client Request
External Load Balancer
or CDN (Cloud Provider)
API Gateway
Kong, Apigee, AWS API GW,
NGINX, Traefik Hub, etc.
Routes / APIs
(Versioning, Transform, Policies)
Backends
(Multiple Clusters / Services /
External Systems)
SCOPE
Multiple Cluster, External Backends
Auth
Caching
WAF
Rate
Limiting
Analytics
Transform
Kubernetes
Cluster A
Kubernetes
Cluster B
External
Backends
Service A
Service B
Microservices
Legacy
Systems
Serverless
(Lambda)
Pods
Pods
Third-party
APIs

Kubernetes Troubleshooting Checklist

Vyomanant academy

When something breaks in Kubernetes, don't randomly restart Pods. Follow a structured process.

1. Check Pod Status

```
kubectl get pods
kubectl get pods -A
kubectl get pods -o wide
```

Look for:

- Pending
- CrashLoopBackOff
- ImagePullBackOff
- ErrImagePull
- OOMKilled
- Running but not working

2. Describe the Pod

```
kubectl describe pod <pod-name>
```

Check:

- Events
- Failed probes
- Scheduling problems
- Image errors
- Volume mounting
- Resource issues

Very important:
Check the Events section.

3. Check Application Logs

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl logs -f <pod-name>
```

For multi-container Pods:

```
kubectl logs <pod-name> -c <container-name>
```

--previous is useful when a container has already crashed and restarted.

4. Check Nodes

```
kubectl get nodes
kubectl describe node <node-name>
```

Look for:

- NotReady
- CPU pressure
- Memory pressure
- Disk pressure
- Scheduling problems

5. Check Services

```
kubectl get svc
kubectl describe svc <service-name>
```

Verify:

Service → Selector → Pod Labels → Endpoints

If the Service has no endpoints, check whether its selector matches the Pod labels.

6. Check Endpoints

```
kubectl get endpoints
kubectl get endpointslices
```

This helps answer:

"Is my Service actdally connected to Pods?"

7. Check Ingress / Gateway

```
kubectl get ingress
kubectl describe ingress <ingress-name>
```

Also check your Gateway API resources if you're using Gateway API.

Verify:

Internet → Load Balancer → Ingress / Gateway → Service → Pod

8. Check Resources

```
kubectl top pods
kubectl top nodes
```

Look for:

- High CPU
- High memory
- OOM situations
- Node resource exhaustion

Also inspect:

```
kubectl describe pod <pod-name>
```

for CPU/memory requests and limits.

9. Check Storage

```
kubectl get pv
kubectl get pvc
kubectl describe pvc <pvc-name>
```

Check for:

- Pending PVC
- Volume mount failures
- Storage capacity
- Incorrect StorageClass

10. Check ConfigMaps & Secrets

```
kubectl get configmaps
kubectl get secrets
kubectl describe configmap <name>
kubectl describe secret <name>
```

Check whether:

- Required configuration exists
- Environment variables are correct
- Secret references are correct
- The application received the expected configuration.

Avoid exposing secret values while troubleshooting.

11. Check Events

```
kubectl get events
kubectl get events -n <namespace>
```

Events can reveal:

- Scheduling failures
- Image-pull errors
- Failed mounts
- Probe failures
- Evictions
- Resource problems

12. Troubleshooting Flow

PROBLEM → Check Status → Describe Resource → Check Logs → Check Events → Check Resources → Check Network / Service → Check Storage / Config → FIND ROOT CAUSE → FIX → VERIFY

13. Common Problems → First Checks

Problem	First checks
Pod Pending	describe pod → Events → node resources
CrashLoopBackOff	logs → logs --previous → describe
ImagePullBackOff	Image name → tag → registry → credentials
Pod OOMKilled	Memory usage → limits → application
Pod Running but unreachable	Service → endpoints → ports → network
Service has no endpoints	Service selector → Pod labels
Node NotReady	describe node → conditions → events
PVC Pending	PVC → StorageClass → PV
DNS failure	CoreDNS → Pod DNS → Service name
High CPU	top pods → application/process → scaling
Ingress failure	Ingress/Gateway → controller → Service → endpoints

14. Easy Memory Trick

STATUS → DESCRIBE → LOGS → EVENTS → RESOURCES → NETWORK → STORAGE → FIX

Don't ask
"How do I restart it?" first.
Ask "Why did it fail?"

15. One-Sentence Summary

Kubernetes troubleshooting is a systematic process of checking status, details, logs, events, resources, networking, storage, and configuration to identify and fix the root cause.

www.vyomanant.com | academy@vyomanant.com | +91 9704286195

Before You Learn Kubernetes, Understand This...

Vyomanant academy

Kubernetes becomes much easier when you understand containers first.
Many beginners jump directly into Pods, Deployments, Services, and Ingress without understanding what Kubernetes is actually managing.

1. Understand Containers

First understand:

Docker Image
↓
Container
↓
Application

Know:

- What is a container?
- What is an image?
- How to build an image
- How to run a container
- Ports
- Volumes
- Container networking

Kubernetes manages containers, so understand containers first.

2. Understand Why Kubernetes Exists

With one container:

1 Container
↓
Easy to manage manually

But with many containers:

100 Containers
10 Servers
Multiple Applications
Different Environments
High Traffic
Failures
↓
Managing everything manually becomes difficult.

Kubernetes helps automate this management.

3. Understand the Kubernetes Mental Model

Think:

Kubernetes Cluster
↓
Nodes
↓
Pods
↓
Containers
↓
Applications

Simple meaning:

- Cluster → Complete Kubernetes environment
- Node → Machine running workloads
- Pod → Smallest deployable Kubernetes unit
- Container → Runs your application

Kubernetes manages containers on multiple nodes as a cluster.

4. Understand Desired State

You tell Kubernetes:
"I want 3 replicas of my application."

Desired State
↓
3 Pods

If one Pod crashes:

3 Pods
↓
1 Pod fails
↓
Kubernetes detects difference
↓
Creates replacement
↓
3 Pods again

Kubernetes continuously works toward your desired state.

5. Understand Networking

Before learning Services and Ingress, understand:

Pod IP
↓
Service
↓
DNS
↓
Port
↓
Load Balancer

Key idea:

Pod IPs can change.
Services provide a stable way to reach Pods.

6. Understand YAML

Kubernetes is heavily configuration-driven.
You should be comfortable reading:

```
apiVersion:
kind:
metadata:
spec:
```

- Don't try to memorize every field.
- Learn how to read and understand the structure.

YAML tells Kubernetes what you want (your desired state).

7. Best Learning Order

1. Linux
2. Networking Basics
3. Docker
4. Docker Compose
5. Kubernetes Architecture
6. Pods
7. Deployments
8. Services
9. ConfigMaps & Secrets
10. Volumes
11. Probes
12. Ingress / Gateway API
13. Networking
14. Scaling
15. Security
16. Monitoring
17. Troubleshooting

8. Easy Memory Trick

Before Kubernetes, remember:

CONTAINER
↓
NETWORK
↓
YAML
↓
DESIRED STATE

If these four concepts are clear, Kubernetes becomes much less confusing.

9. One-Sentence Summary

Before learning Kubernetes, understand containers, networking, YAML, and the idea of desired state—because Kubernetes is essentially automating the deployment, networking, scaling, and management of containerized applications.

Understand the Basics
Build for the Future

www.vyomanant.com | academy@vyomanant.com | +91 9704286195

Containers Make Modern Apps Possible !

How Docker Images Run inside a Kubernetes Cluster

JyothiMulkuntla

"Build Ship Orchestrate Scale"

Same app Bigger possibilities!

1 Core idea

- Docker image does NOT run directly in Kubernetes.
- Kubernetes schedules a Pod.
- The node's container runtime pulls the image and starts a container.

"Kubernetes doesn't run images, It runs Pods (and the node runs the containers)."

10 Key memory points

- ☆ Image = packaged application
- ☆ Container = running instance of image
- ☆ Pod = Kubernetes unit hosting containers
- ☆ Deployment = manages Pods
- ☆ Prefer versioned image tags or digests, not latest ⚠

Small Concepts Big Impact !

2 Flow (End-to-End)

Dockerfile (app definition) → Docker Image (built locally) → Registry (Docker Hub / ECR / ACR / GAR / GHCR) → Deployment (desired state) → ReplicaSet (manages replicas) → Pod (Pod spec) → Scheduler (finds a node) → Worker Node (runs the pod) → Kubelet (talks to CRI) → containerd / CRI-O (pulls & runs) → Container (running) → Application (your app)

3 Step 1: Build image

- Package app + runtime + libraries + dependencies.
- Example:

```
docker build -t myapp:v1 .
```

Dockerfile

```
FROM python:3.11
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
CMD ["python","app.py"]
```

Defines how to build the image.

4 Step 2: Push to registry

- Push the image to a container registry.
- Example:

```
docker push docker.io/myuser/myapp:v1
```

- Popular registries:

Docker Hub / ECR / ACR / GAR / GHCR / Private registry

Container Registry

Store your image in a registry so Kubernetes can pull it!

5 Step 3: Deploy in Kubernetes

- Deployment references the image.
- Kubernetes creates a ReplicaSet and Pods.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: myapp
        image: myuser/myapp:v1
```

Deployment → ReplicaSet → Pod Pod Pod

You declare the desired state, Kubernetes makes it real!

6 Scheduling

- Scheduler picks a worker node based on:
  - CPU
  - Memory
  - Affinity / anti-affinity
  - Taints / tolerations
  - Resource availability

Scheduler → Node 1 (CPU, RAM) Node 2 (taints) Node 3 (affinity)

Kubernetes scheduler finds the best node for your Pod!

7 On the worker node

- Kubelet talks to container runtime via CRI (Container Runtime Interface).
- Common runtimes: containerd, CRI-O.
- Modern Kubernetes does not require Docker Engine on nodes.

Docker-built OCI images still work

Kubelet ⟷ CRI ⟷ Containerd / CRI-O

Kubelet asks the runtime to pull the image and start the container.

8 What runtime does

- Pull image layers (from registry).
- Create filesystem.
- Configure namespaces & cgroups.
- Set up networking.
- Mount volumes.
- Start app process.

Image (from registry) → Container Runtime (containerd / CRI-O) → Container (running)

Turns the image into a running container !

9 Pod & Networking

- Pod = smallest deployable unit.
- Pod can contain one or more containers.
- Containers in a Pod share network namespace, IP, and volumes.
- Service gives stable access to Pods.

Service (stable IP / DNS) → Pod (shared network namespace, IP, volumes) → Container 1, Container 2

All containers in a Pod share the same network & storage (namespace, IP, volumes).

Service = stable access to a changing set of Pods.

Same Docker image. Bigger things with Kubernetes !

- ☑ Build once
- ☑ Deploy anywhere
- ☑ Scale effortlessly
- ☑ Focus on your app !

JyothiMulkuntla

Keep Learning Keep Building Keep Growing !

Ingress vs Gateway API in Kubernetes

Vyomanant academy

1. Simple Explanation

Both Ingress and Gateway API are used to control how external traffic reaches applications inside Kubernetes.
Ingress is the traditional way to route HTTP/HTTPS traffic, while Gateway API is the newer, more flexible and extensible approach for modern traffic management.

Ingress = Simple entry and routing,
Gateway API = Flexible, extensible, and modern traffic management.

kubernetes

- ☑ Control external traffic to applications
- ☑ Ingress is simple and widely used
- ☑ Gateway API is more flexible and extensible
- ☑ Supports advanced routing capabilities
- ☑ Better role separation with Gateway API
- ☑ Designed for modern Kubernetes use cases

2. Simple Analogy

Think of your Kubernetes cluster as a shopping mall:

Ingress
(Traditional Entrance Desk)



- Fixed set of routing rules
- Simple and straightforward

Like a traditional entrance desk.

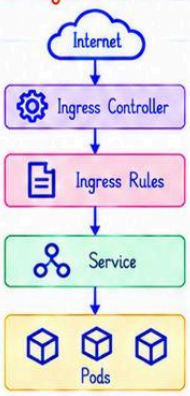
Gateway API
(Modern Entrance System)



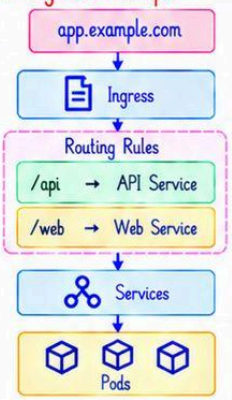
- More flexible
- Richer traffic-routing capabilities
- Separate roles

Like a modern, smart entrance system.

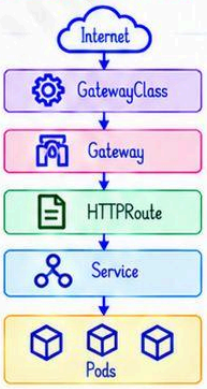
3. Ingress Flow



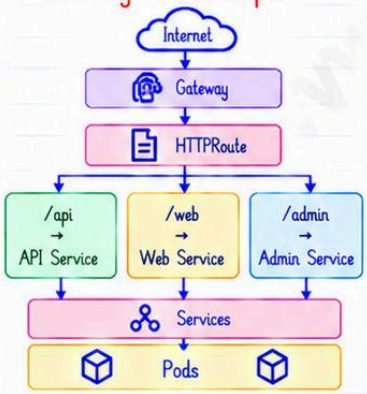
4. Ingress Example



5. Gateway API Flow



6. Gateway API Example



7. Ingress vs Gateway API Comparison

Feature	Ingress	Gateway API
Age	Older	Newer
Main resource	Ingress	Gateway + Routes
Routing	Basic	More expressive
Traffic types	Mainly HTTP/HTTPS	Designed for broader traffic use cases
Role separation	Limited	Stronger
Extensibility	Limited	Designed for extensibility
Configuration	Simpler	More structured
Modern direction	Legacy / common	Modern recommended API family

8. When to Use?

Use Ingress when:

- ☑ You need simple HTTP/HTTPS routing
- ☑ Your cluster already uses an Ingress controller
- ☑ You want a straightforward setup

Use Gateway API when:

- ☑ You need advanced routing features
- ☑ You want better role separation
- ☑ You need a more extensible solution
- ☑ You are building for long-term, modern Kubernetes environments

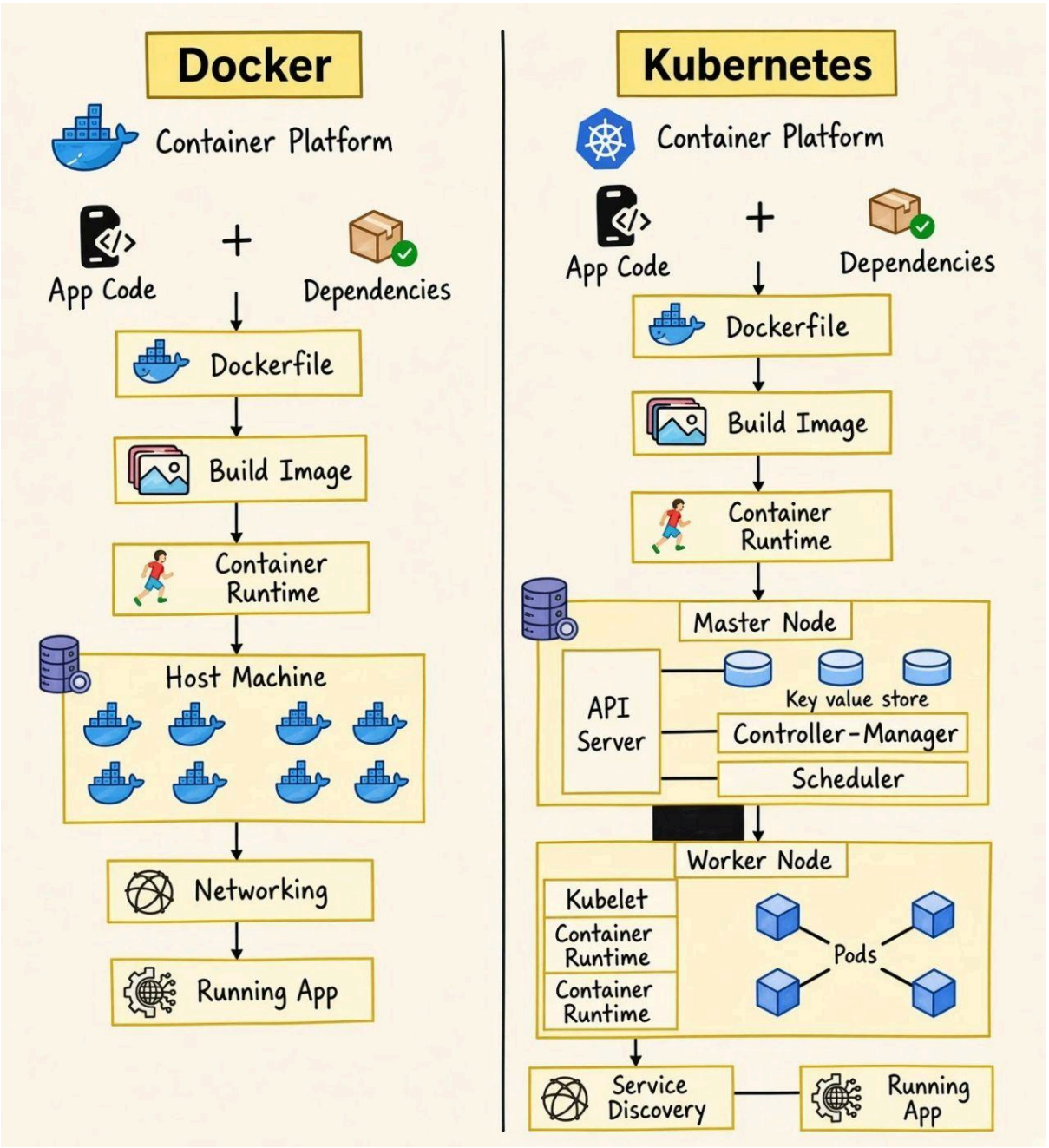
9. Easy Memory Trick

Ingress = Simple entry + routing
Gateway API = Gateway + flexible routes + role separation

10. One-Sentence Summary

Ingress provides a simpler, traditional way to route external HTTP/HTTPS traffic into Kubernetes, while Gateway API provides a more flexible and extensible model for modern traffic management.

www.vyomanant.com | academy@vyomanant.com | +91 9704286195


Docker
Container Platform
App Code
Dependencies
Dockerfile
Build Image
Container Runtime
Host Machine
Networking
Running App
Kubernetes
Container Platform
App Code
Dependencies
Dockerfile
Build Image
Container Runtime
Master Node
API Server
Key value store
Controller-Manager
Scheduler
Worker Node
Kubelet
Container Runtime
Container Runtime
Pods
Service Discovery
Running App

Code → To Cloud

End-to-End DevOps CI/CD Pipeline

Build Test Deploy Repeat

Developer	Developer writes code and commits
GitHub	Code pushed to GitHub
Jenkins	Jenkins triggers the pipeline
Build & Test	Build the code and run unit tests
SonarQube	Check code quality and bugs
OWASP	Scan for dependency vulnerabilities
Docker Build	Build Docker image
Trivy Scan	Scan image for vulnerabilities (CVEs)
Amazon ECR	Push image to Amazon ECR
Update Helm values.yaml	Update image tag in values.yaml
GitHub	Commit and push changes
Argo CD	Argo CD detects changes and syncs the cluster
Kubernetes	Application deployed to Kubernetes
Production	Application is live to Kubernetes

Automate Everything!

Better Faster Safer

DevOps = Happy Users

What I Learned

Building this pipeline taught me that DevOps is not just about deployment.

It's about

- Automation
- Security
- Observability
- Reliability
- Repeatability

Best Practices

- Automate every deployment.
- Fail early when quality or security checks fail.
- Store infrastructure and deployment configs in Git.
- Monitor everything and alert proactively.

Question for You

Which stage do you think adds the most value in a CI/CD pipeline?

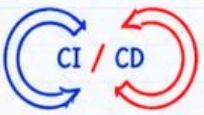
Key Takeaways

- Automation First
- Shift Left Security
- GitOps Deployment
- Continuous Delivery

CI vs CD vs Continuous Deployment

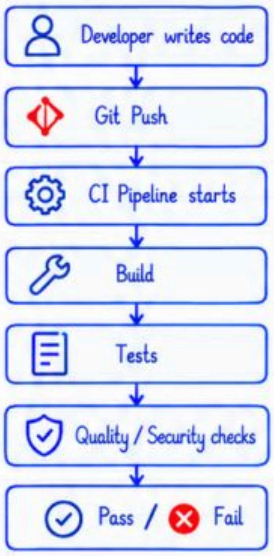
Anusha Janjirala

The easiest way to understand them is:
Code → Build → Test → Release → Deploy
CI/CD is the process of automating these steps.



1. CI — Continuous Integration

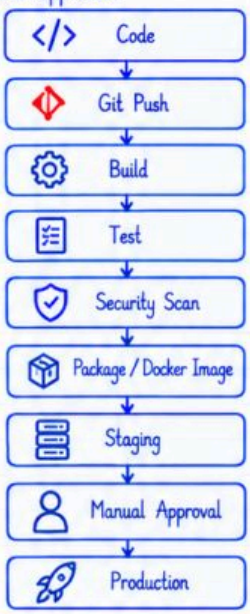
CI means developers frequently merge code into a shared repository, and every change is automatically built and tested.



Main goal of CI:
Find problems early before they reach production.

2. CD — Continuous Delivery

Continuous Delivery means the application is automatically built, tested, and prepared for release, but production deployment usually requires approval.



Everything is ready to deploy, but a human decides when to release it.

3. Continuous Deployment

Continuous Deployment means every change that successfully passes the automated pipeline is automatically deployed to production.

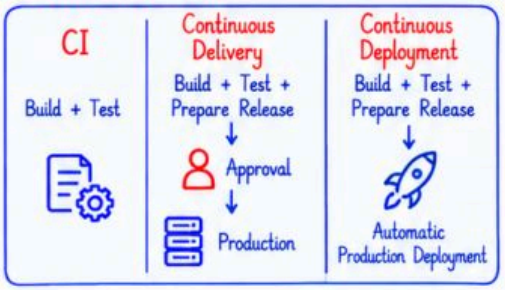


No manual approval.
Every successful change is automatically deployed to production.

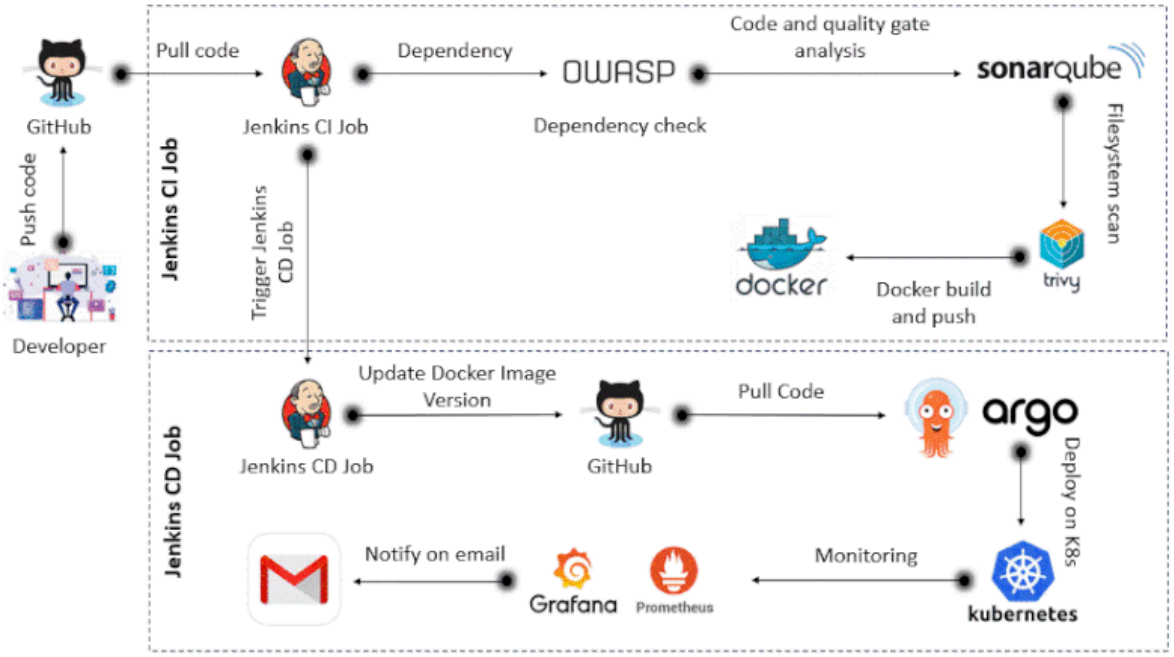
4. Key Differences

Concept	What happens?
CI	Code is built and tested automatically
Continuous Delivery	Code is automatically prepared and ready for production, but deployment may need approval
Continuous Deployment	Successful changes automatically go to production

5. Remember This



www.vyomanant.com | academy@vyomanant.com | +91 9704286195


GitHub
Pull code
Jenkins CI Job
Dependency
OWASP
Dependency check
Code and quality gate analysis
sonarqube
Filesystem scan
Push code
Developer
Jenkins CI Job
Trigger Jenkins CD Job
docker
Docker build and push
trivy
Jenkins CD Job
Update Docker Image Version
Jenkins CD Job
GitHub
Pull Code
argo
Deploy on K8s
Notify on email
Grafana
Prometheus
Monitoring
kubernetes